

Programación II: Laboratorio 2

Diego Hernández

9 de octubre de 2025

1 Introducción

El siguiente informe resume contenidos introductorios a la Programación Orientada a Objetos (POO), con énfasis en el lenguaje Python.

La Programación Orientada a Objetos (POO) es un paradigma de programación en el que los conceptos del mundo real relevantes para nuestro problema se modelan a través de clases y objetos, y en el que nuestro programa consiste en una serie de interacciones entre estos objetos.[1]

2 Herencia y Polimorfismo

2.1 Herencia

En la POO, la **herencia** es el proceso mediante el cual una subclase hereda atributos y métodos de una súperclase o clase padre.

2.1.1 Herencia múltiple y MRO

Algo particular que ocurre con la herencia es que puede ocurrir la herencia múltiple, es decir, que una subclase herede métodos y atributos de más de una súperclase, ya sea porque hereda de dos súperclases de igual jerarquía o porque hereda de una súperclase que a su vez hereda de una súperclase aún mayor en jerarquía.

En dado caso, si se invoca algún método de una clase y esta no lo tiene en sí, sino que es parte de la herencia de sus súperclases, el orden en que debe recorrer las súperclases está definido por el **MRO** (*Method Resolution Order*), el cual dictamina qué orden debe recorrer las clases padre para buscar los métodos solicitados.

En el caso de Python, este lenguaje a partir de su versión 2.3 usa un MRO basado en el algoritmo *C3 linearization*, el cual básicamente hace que se revisen las clases inmediatamente superiores a la subclase a la que se le pide el método, y luego seguir con las superiores de un nivel mayor. Esto difiere del MRO que usaba antiguamente algoritmo *DFS*, el cual hacía que la búsqueda siempre terminara en la súperclase de mayor nivel cada vez que pudiese, lo cual generaba repeticiones innecesarias.

2.2 Polimorfismo

Relacionado al concepto de herencia, una vez heredados los métodos y atributos en la subclase, estos pueden ser sobreescritos con características que al programador le parezca pertinente redefinir para esa subclase en particular, y estos métodos y atributos podrán conservar el mismo nombre. A esto se le llama **polimorfismo**, ya que el método o atributo es el mismo, pero tiene un funcionamiento interno (o forma) distinto.

Sin embargo, esto no es exclusivo de las relaciones de herencia: en el concepto de programación conocido como *Duck Typing*, los métodos y atributos pueden llamarse igual en clases distintas y sin relación jerárquica, y tener formas distintas, y esto aún sería polimorfismo.

En resumen, el polimorfismo puede darse en todo tipo de relaciones entre clases (herencia o clases no asociadas), mientras el método o atributo se llame igual y tenga un funcionamiento distinto.

3 Abstracción: Clases abstractas e Interfaces

La **abstracción** es un término de programación y POO que hace la ocultación de la complejidad intrínseca de una aplicación al exterior, centrándose sólo en como puede ser usada, lo que se conoce como interfaz.[3]

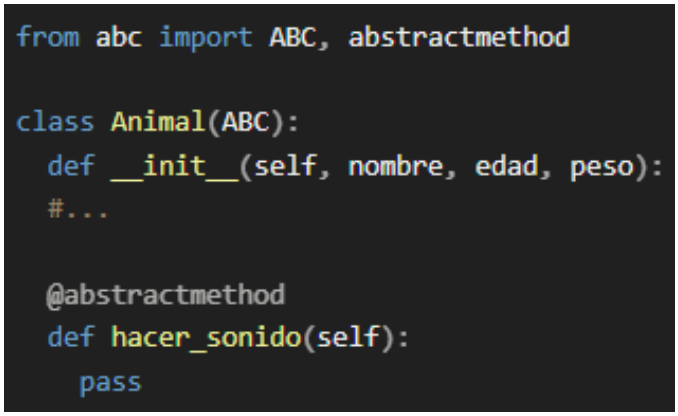
En Python, hay formas de materializar o escribir esta abstracción a la hora de desarrollar código orientado a objetos, diseñando clases generales de las que se desprendan clases (en este sentido, subclases) con características similares entre sí, fijando atributos y métodos que deben estar sí o sí implementadas en estas subclases, pero no se define su código. Podría decirse que son planos de construcción de clases.

Hay dos modos principales en Python de llevar a cabo esta abstracción con clases generales: las clases abstractas y las interfaces.

3.1 Clases abstractas

Las clases abstractas se caracterizan porque permiten métodos concretos con código y se transfiere a las subclases; es decir, no es completamente abstracta.

En Python, usan el módulo *abc* para obligar a implementar en las subclases los métodos que se declaran como métodos abstractos.



```python
from abc import ABC, abstractmethod

class Animal(ABC):
  def __init__(self, nombre, edad, peso):
  #...

  @abstractmethod
  def hacer_sonido(self):
    pass
```

Figure 1: Muestra de una clase abstracta.

3.2 Interfaces

Las interfaces se diferencian de las clases abstractas en el sentido en que estas sí son completamente no implementables.

En Python, se usan los *Protocols* del módulo *typing* para verificar que un objeto cumple con el diseño de la interfaz.

- En primer lugar, se diseña una interfaz y se le asigna un verificador.
- En segundo lugar, se crea una clase y se instancia un objeto de esa clase.
- Finalmente, se valida con una función si el objeto cumple con el diseño de la interfaz. Es decir, se valida el objeto en vez de clase, a diferencia de lo que sucedía con las clases abstractas.

```python
# Creación de clases
from typing import Protocol, runtime_checkable

@runtimecheckable
class FiguraGeometrica(Protocol):
  def area(self) -> float:
    ...

class Cuadrado:
  def __init__(self, lado: float)
    self.lado = lado

  def area(self) -> float:
    return self.lado ** 2

cuadrado = Cuadrado(4)

# Verificación de cumplimiento de protocolo
print(isinstance(cuadrado, FiguraGeometrica))
```

Figure 2: Ejemplo de interfaz con *Protocols*.

Bibliografía

[1] González Duque, R. (2010). *Python para Todos*.

[2] El Libro de Python (s.f.). Herencia en Python. https://ellibrodepython.com/herencia-en-python

[3] El Libro de Python (s.f.). Abstracción en programación. https://ellibrodepython.com/abstraccion-en-programacion